CSCI378 – Data Structures and Algorithms
Assignment 1 - Solution

Problem 1 (Graded)

```
import java.util.*;
public class Application {

   // 60 pts for the mergeUnsortedLists method
   public static LinkedList mergeUnsortedLists(LinkedList list1, LinkedList list2) {

         LinkedList list3 = new LinkedList();  // 5 pts
         Node curr1 = list1.getHead(), curr2 = list2.getHead(); // 5 pts
         while (curr1 != null && curr2 != null) // 6 pts
         {
                 list3.addTail(curr1.data); // 6 pts
                 curr1 = curr1.next; // 2 pts
                 list3.addTail(curr2.data); // 6 pts
                 curr2 = curr2.next; // 2 pts
         } // end while

         // In case the first list is longer (it still has nodes)
         while (curr1 != null) // 5 pts
         {
              list3.addTail(curr1.data); // 6 pts
              curr1 = curr1.next; // 2 pts
         } // end while

           // In case the second list is longer (it still has nodes)
         while (curr2 != null) // 5 pts
         {
              list3.addTail(curr2.data); // 6 pts
              curr2 = curr2.next; // 2 pts
         } // end while

         return list3; // 2 pts

    } // end method

  // 20 pts for the main method
  public static void main(String[] args) {

         int n;
         NodeData nd;
         Scanner scan = new Scanner(System.in);
         LinkedList list1 = new LinkedList();
         LinkedList list2 = new LinkedList();

         // Fill the first list
         System.out.print("Enter the values of the first list (and 0 to stop):");
         n = scan.nextInt();
         while (n != 0) {
             nd = new NodeData(n);
             list1.addTail(nd);
             n = scan.nextInt();
         }
```

```
        // Fill the second list
        System.out.print("Enter the values of the second list (and 0 to stop):");
        n = scan.nextInt();
        while (n!=0) {
            nd = new NodeData(n);
            list2.addTail(nd);
            n = scan.nextInt();
        }

        // Merge the two lists
        LinkedList list3 = mergeUnsortedLists(list1, list2);

        // output the three lists
        System.out.println("First list: ");
        list1.print();
        System.out.println("Second list: ");
        list2.print();
        System.out.println("Merged list: ");
        list3.print();

  }// end main

} // end Application

// 10 pts for the use of meaningful names and the documentation (use of comments).

// 10 pts for the correctness (no errors) and completeness (program does all what
it is supposed to do).
```

Problem 2 (Ungraded)

```
import java.util.*;
public class Application {

  public static LinkedList multiply (LinkedList l1) {

    LinkedList l2 = new LinkedList();
    Node temp1 = l1.getHead(), temp2;

    while(temp1 != null)
    {
        temp2 = l1.getHead();
        int product = temp1.data.value;
        while(temp2 != temp1)
        {
             product *= temp2.data.value;
             temp2 = temp2.next;
        }
        NodeData data = new NodeData(product);
        l2.addTail(data);
        temp1 = temp1.next;
    }
    return l2;
  }
```

```
  public static void main(String[] args) {
        NodeData nd;
        int n;
        Scanner scan = new Scanner(System.in);
        LinkedList list1 = new LinkedList();

        // Fill the first list
        System.out.print("Enter the values of the first list (and 0 to stop):");
        n = scan.nextInt();
        while (n != 0) {
            nd = new NodeData(n);
            list1.addTail(nd);
            n = scan.nextInt();
        }

        // Call multiply
        LinkedList list2 = multiply(list1);

        // output list1
        System.out.println("First list: ");
        List1.print();

        // output list2
        System.out.println("Second list: ");
        list2.print();

  }// end main

} // end Application
```

Problem 3 (Ungraded)

```
import java.util.*;
public class Application {

  public static boolean isSubset(LinkedList a, LinkedList b){
       //get the number of nodes in each list
        int aLength = a.countNodes(), bLength = b.countNodes(), pos;
       //create a copy of the largest list because we need to make changes
       // to the copy while the initial list remains intact
        LinkedList smallest = a, largest = b.copy();
        if(bLength < aLength)
        {
            smallest = b;
            largest = a.copy();
        }
        //for each node in the smallest, search for it in the largest
        Node curr1 = smallest.getHead(), curr2;
        while(curr1 != null)
        {
            // if not found then not a subset
            if(!largest.search(curr1.data))
                return false;
```

```
            // if found delete it from the largest, to make sure if a
            // value occurs more than once then all occurrences are taken
            // into consideration
            pos = 1;
            curr2 = largest.getHead();
            while (curr2 != null) // find pos of curr1.data in largest
            {
              if (curr2.data.compareTo(curr1.data) == 0) // found
                  break;
              curr2 = curr2.next;
              pos++;
            }
            largest.delete(pos);  // delete the value of curr2.data
            curr1 = curr1.next;
        }
        //if all elements were found then the smallest is a subset
        return true;
 }

  public static void main(String[] args) {

        int n;
        NodeData nd;
        Scanner scan = new Scanner(System.in);
        LinkedList list1 = new LinkedList();
        LinkedList list2 = new LinkedList();

        // Fill the first list
        System.out.print("Enter the values of the first list (and 0 to stop):");
        n = scan.nextInt();
        while (n != 0) {
            nd = new NodeData(n);
            list1.addTail(nd);
            n = scan.nextInt();
        }

        // Fill the second list
        System.out.print("Enter the values of the second list (and 0 to stop):");
        n = scan.nextInt();
        while (n!=0) {
            nd = new NodeData(n);
            list2.addTail(nd);
            n = scan.nextInt();
        }

        // Call isSubset
        if (isSubset(list1, list2))
            System.out.println("One of the two lists is a subset of the other.");
        else
            System.out.println("None of the two lists is a subset of the other.");

   }// end main

} // end Application
```

Problem 4 (Ungraded)

```
public class LinkedList {

   // This method should be added to the class LinkedList
   public void swap(int i, int j) {
        int size = countNodes();
        if(i > size || j > size || i < 1 || j < 1){ // check if i and j are valid
            System.out.println("Invalid values of arguments!");
            return;
        }
        if(i != j) {
            Node p1 = head, p2 = head;
            for(int i1 = 1; i1 < i; i1++) // find the node at position i
                p1 = p1.next;
            for(int j1 = 1; j1 < j; j1++) // find the node at position j
                p2 = p2.next;
            //swap the values
            int temp = p1.data.value;
            p1.data.value = p2.data.value;
            p2.data.value = temp;
        }
   }

import java.util.*;
public class Application {

  public static void main(String[] args) {
        NodeData nd;
        int n;
        Scanner scan = new Scanner(System.in);
        LinkedList l1 = new LinkedList();

        // Fill the first list
        System.out.print("Enter the values of the list (and 0 to stop):");
        n = scan.nextInt();
        while (n != 0) {
            nd = new NodeData(n);
            l1.addTail(nd);
            n = scan.nextInt();
        }

        // output the list elements before the swap
        System.out.println("Before swap the list elements are: ");
        l1.print();

        // Call swap
        l1.swap(1,2);

        // output the list elements after the swap
        System.out.println("After swap the list elements are: ");
        l1.print();

  }// end main

} // end Application
```